

Rudra Techno

A SaaS platform that designs and builds custom websites & software. Clients request projects, track delivery in real time, and pay online — with role-based dashboards for clients, admins, and developers.

Next.js 16

React 19

Tailwind CSS v4

PostgreSQL + Prisma

Auth.js

Razorpay + Stripe

Vercel

Owner: Anuj Rastogi · Document: aboutmywebsite · Generated 2026-06-08

What this document covers: a brief, plain-language guide to every page on the website, the backend that powers it, the database design, and how online payments work through Razorpay (with Stripe for global clients). Architecture diagrams are included throughout.

1. How the platform works

Rudra Techno has **two halves**: a public *marketing website* that attracts clients, and a private *application* (the dashboards) that runs the actual work. A visitor browses services and pricing, signs up, and is taken into a dashboard chosen by their **role**:

- CLIENT** requests projects, tracks progress, chats, and pays invoices — sees only their own data.
- ADMIN** manages every project, assigns developers, sets pricing, and watches revenue.
- DEVELOPER** sees only assigned tasks on a kanban board and uploads deliverables.

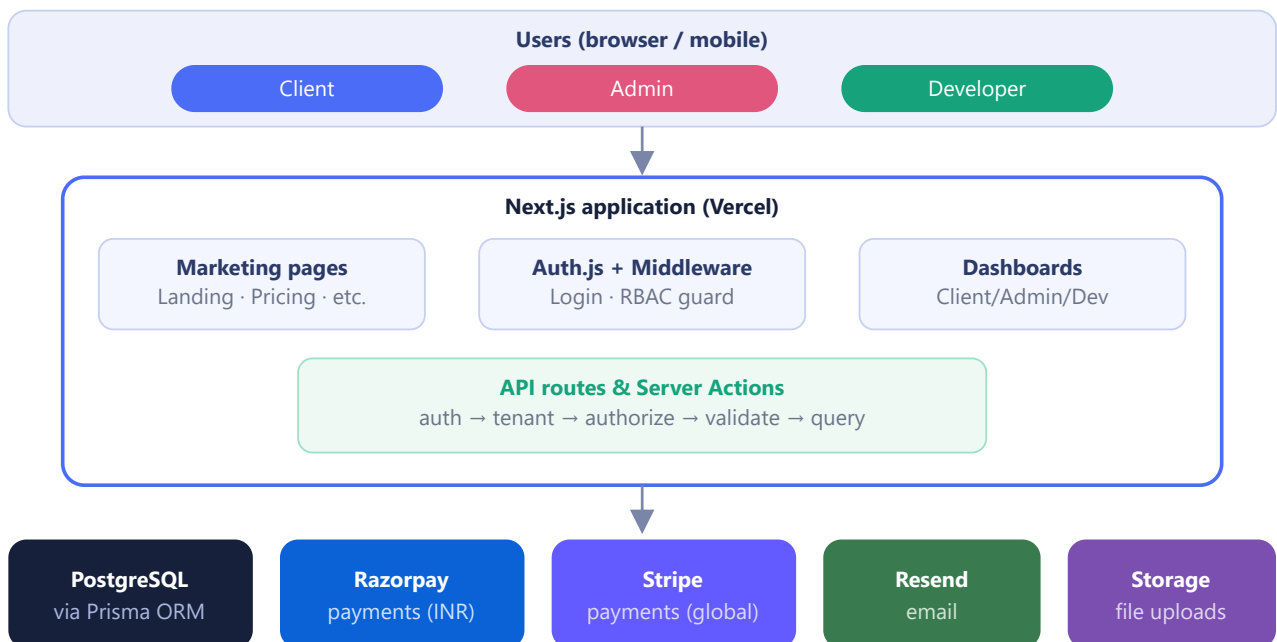


Figure 1 — High-level system architecture: users → Next.js app → database & external services.

2. Every page on the website

Public marketing pages (anyone can view)

Page	Path	What it does
Landing	/	Hero, features, video, testimonials, pricing, blog, contact — the main sales page.
Services	/services	The six services offered (websites, web apps, mobile, SaaS, UI/UX, support).
Portfolio	/portfolio	Case studies of past projects and their results.
Pricing	/#pricing	Plans and a "request custom quote" call-to-action.
Blog	/blog , /blog-details	Articles and updates.
About	/about	Company story and team.
Support / Contact	/contact	Contact form and support entry point.

Account pages (sign in & out)

Page	Path	What it does
Sign In	/signin	Email + password or Google login.
Sign Up	/signup	Create an account (starts as a Client).
Forgot / Reset password	/forgot-password	Email a secure reset link.
Verify email	/verify-email	Confirm the email address after signup.

Application pages (login required, role-gated)

Role	Page	What it does
CLIENT	Projects · New request · Project detail	Request a build via a step-by-step wizard; track its timeline and progress.
CLIENT	Messages · Invoices · Tickets · Settings	Chat with the team, pay invoices online, raise support tickets.
ADMIN	Overview · Projects · Users	See KPIs, manage all projects, invite users and set roles.
ADMIN	Packages · Tickets · Analytics	Control pricing, handle the support queue, view revenue analytics.
DEVELOPER	Tasks (kanban) · Project detail	See only assigned work; drag tasks across Backlog → In Progress → Review → Done.
DEVELOPER	Uploads · Settings	Upload deliverables to assigned projects.

Status: the marketing and account pages exist today (rebranded to Rudra Techno). The application dashboards, database wiring, and payments are being built in phases — see the architecture plan in `docs/ARCHITECTURE.md` .

3. The database

The app uses **PostgreSQL** (a reliable relational database) accessed through **Prisma**, which turns database tables into easy-to-use code. It is **multi-tenant**: every record carries a `tenantId` so one company's data is always isolated from another's.

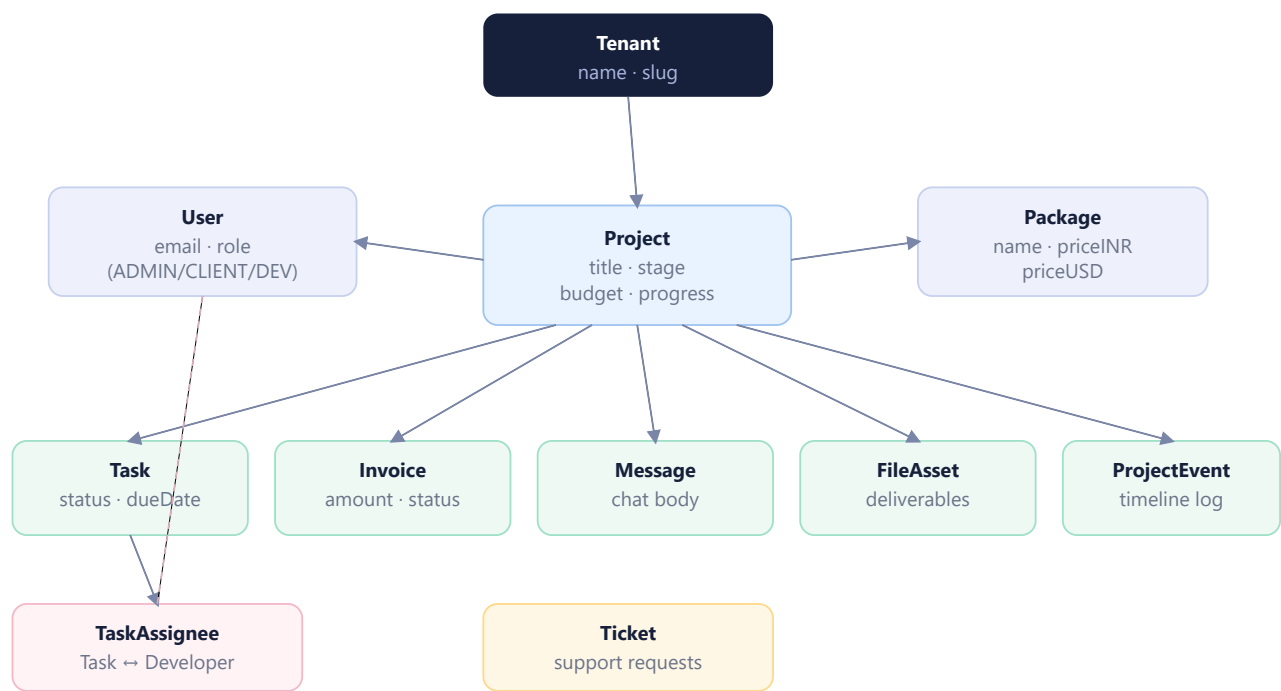


Figure 2 — Database relationships (ERD). A Tenant owns Users, Projects and Packages; each Project owns its Tasks, Invoices, Messages, Files and timeline Events.

Main tables in plain English

Table	Holds
Tenant	The organisation. Keeps every company's data separate (multi-tenancy).
User	People who log in, each with a role: Admin, Client, or Developer.
Package	Pricing packages, with separate INR and USD prices.
Project	A client's build request — its stage, budget, deadline, and % progress.
Task / TaskAssignee	Units of work on a project and which developer is assigned.
Invoice	A bill — amount, currency, status, and which payment provider was used.
Message	Chat messages between client and team on a project.
Ticket	Support requests.
FileAsset	Uploaded files / deliverables.
ProjectEvent	An activity log that powers the client's progress timeline.
Account / Session	Used by the login system (Auth.js) to keep users signed in.

4. The backend & security

There is no separate server to manage — the backend lives **inside the same Next.js app** as API routes and "server actions". Every request that touches data passes through the same five safety checks before anything happens:

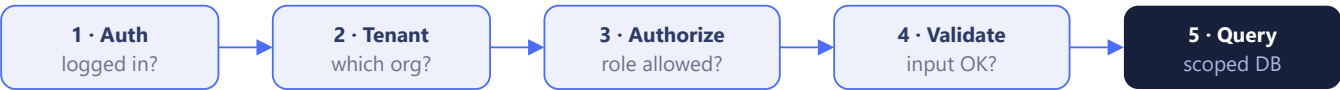


Figure 3 — Every backend request runs the same pipeline before reading or writing data.

Role-based access (RBAC)

Access is enforced in three layers so the UI can never be tricked:

- **Route guard** — middleware blocks wrong-role URLs.
- **API rules** — a central policy decides each action.
- **UI hiding** — buttons appear only if allowed.

Login flow (Auth.js)

- Sign up → password is hashed, email verified.
- Sign in → a secure session token is issued.
- The token carries the user's id, tenant and role.
- Every page re-checks the token on the server.

Example permissions

Action	Admin	Client	Developer
Create a project request	✓	✓ (own)	—
View all projects	✓	—	—
Assign developers	✓	—	—
Update task status	✓	—	✓ (assigned)
Pay an invoice	✓	✓ (own)	—
Manage users & roles	✓	—	—

5. Online payments — Razorpay (and Stripe)

Invoices are paid online. Indian clients pay in INR through **Razorpay**; international clients pay through **Stripe**. Both sit behind one shared "payments" layer, so the rest of the app does not care which is used — it just asks for an order and waits for confirmation.

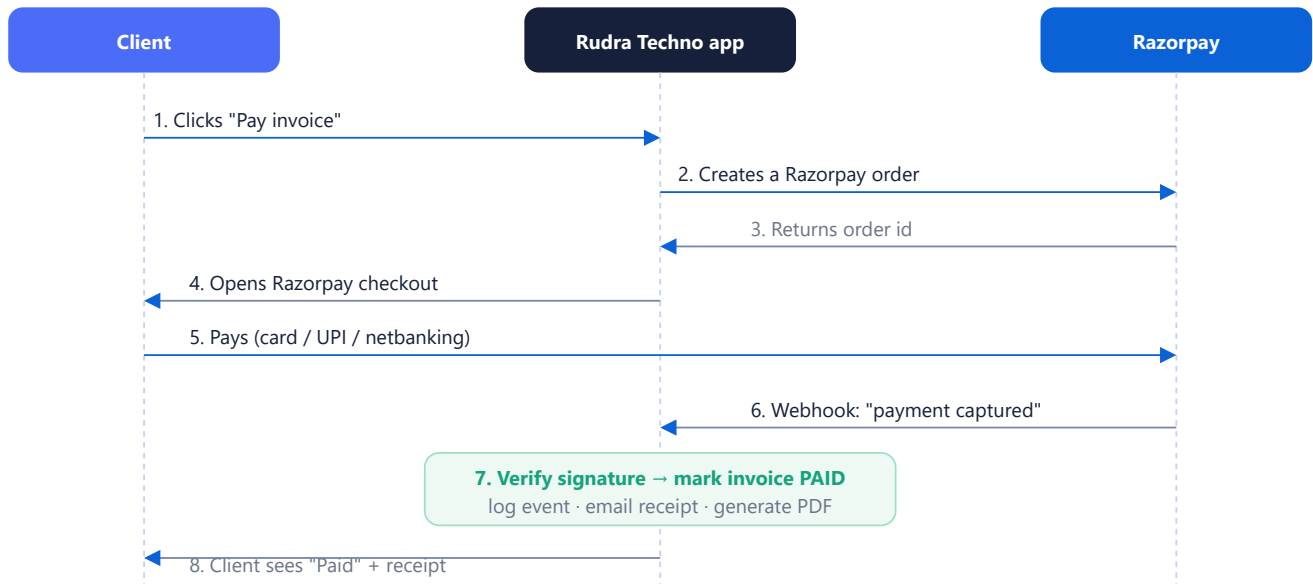


Figure 4 — Razorpay payment flow. The webhook + signature check is what actually confirms payment, so it can't be faked from the browser.

Why a webhook, not just the browser?

The browser saying "payment done" is not trusted on its own. Razorpay separately calls the server (a *webhook*) with a signed message; the server verifies that signature with a secret key before marking the invoice **PAID**. This prevents fraud and handles cases where the user closes the tab after paying.

Provider	Used for	Methods
Razorpay	Indian clients (INR)	UPI, cards, netbanking, wallets
Stripe	International clients	Cards, wallets (USD/EUR...)

6. Technology summary

Layer	Technology
Website & backend	Next.js 16 (App Router), React 19, TypeScript
Design	Tailwind CSS v4, dark/light themes
Database	PostgreSQL + Prisma ORM
Login	Auth.js (NextAuth) — email/password + Google
Payments	Razorpay (India) + Stripe (global)
Email	Resend
Hosting	Vercel (app) + Neon/Supabase (database)

